

Comunicação entre processos

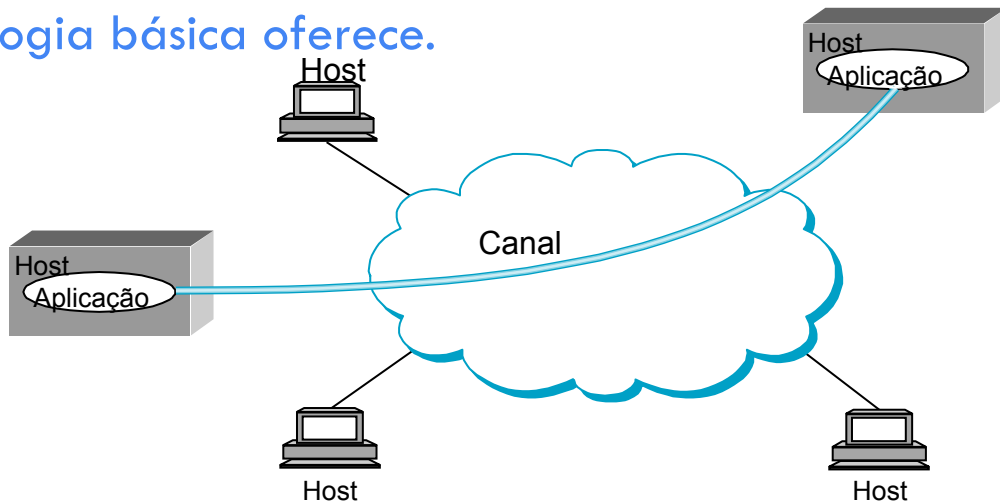
irineu.sotoma@ufms.br

Irineu Sotoma

Funcionalidade da rede

Comunicação entre processos

- Transforma conectividade host-a-host em comunicação processo-a-processo.
- Preenche a lacuna entre o que as aplicações esperam e o que a tecnologia básica oferece.



Suporte a serviços comuns

□ Os requisitos de aplicações variam:

- Uma transferência de arquivo usando FTP
- Uma transferência de *webpage* usando HTTP
- Um fluxo de vídeo (*video streaming*) usando RTP

□ Requisitos:

- Confiabilidade total, ou
- Restrições de tempo: atraso e *jitter*

Programação com sockets

4

Meta: aprender a construir aplicações cliente/servidor que se comunicam usando sockets

API Sockets

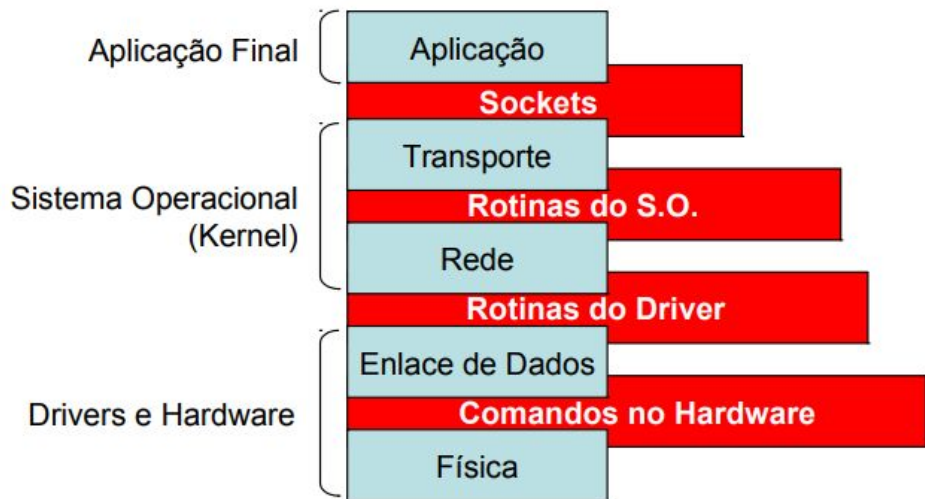
- ❑ apareceu no BSD4.1 UNIX em 1981
- ❑ são explicitamente criados, usados e liberados por apls
- ❑ paradigma cliente/servidor
- ❑ dois tipos de serviço de transporte via API Sockets
 - Datagrama, não confiável
 - fluxo de bytes, confiável

socket
uma interface (uma “porta”), local ao hospedeiro, criada por e pertencente à aplicação, e controlado pelo SO, através da qual um processo de aplicação pode tanto enviar como receber mensagens para/de outro processo de aplicação (remoto ou local)

Programação Socket

API Sockets

- desenvolvida na época da implementação dos protocolos TCP/IP pelo grupo da UC, Berkeley
- elemento de ligação entre a aplicação e um sistema de mais baixo nível.



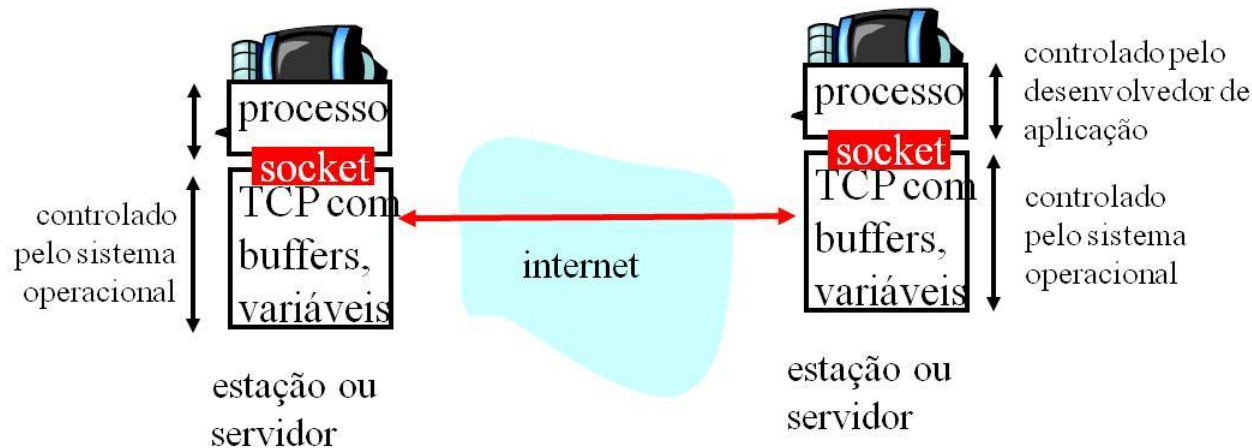
Programação com sockets

Socket: uma porta entre o processo de aplicação e um protocolo de transporte fim-a-fim (UDP ou TCP)

Serviço TCP: transferência **confiável** de **bytes** de um processo para outro

Sem estrutura, ou
qualquer inf. semântica

Serviço UDP: transferência **não confiável** de **datagramas**



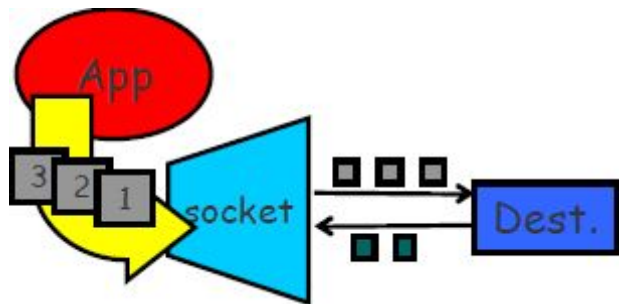
O que é um socket?

- ❑ Interface padrão do sistema Unix em C
 - Implementada também no Windows (*Winsock*)
- ❑ Na camada de transporte, os sockets suportam principalmente os protocolos:
 - TCP (*Transmission Control Protocol*)
 - Orientado a conexão, para *stream* (fluxo de bytes), confiável
 - UDP (*User Datagram Protocol*)
 - para datagramas – sem garantias
 - Mais recentemente, também o SCTP (*Stream Control Transmission Protocol*) (em C) (RFC 6458)
 - A interface de sockets teoricamente permite acesso para outras famílias de protocolos

Dois tipos principais de sockets

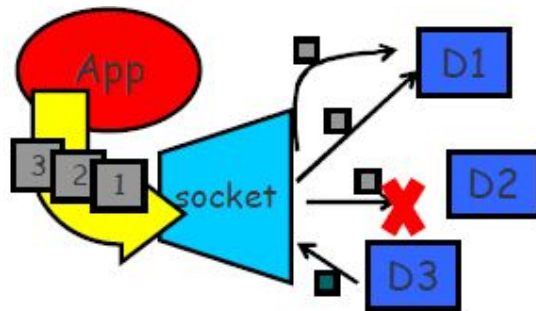
□ TCP (SOCK_STREAM)

- entrega confiável
- entrega em ordem
- orientado à conexão
 - conexão contínua entre o par de comunicação
- bidirecional



□ UDP (SOCK_DGRAM)

- entrega não confiável
- ordem não garantida
- Não há noção de conexão – aplicação indica destinatário para cada datagrama
- envio e recebimento



Visão Socket da Internet

- ❑ Ao chamar uma função que cria um socket, a função cria uma interface, que é acessível via um manipulador (*file descriptor* em C)
- ❑ Além do protocolo TCP ou UDP, existem duas coisas que um *peer* (cliente ou servidor) precisa saber sobre a máquina que deseja se comunicar: o endereço IP (4bytes) (há funcionalidades para permitir IPv6 (RFC 3493) também) e a **porta**.
- ❑ Uma vez que os campos de porta em TCP e UDP têm 16 bits (2 bytes), os números de porta variam de 0 a 65535.
 - Esse espaço de numeração é considerado separadamente em cada protocolo de transporte.

Portas

- ❑ Número das portas (RFC 6335):
 - 0-1023: reservado, deve ser root (“Well Known Ports”)
 - 1024 – 49151: pode ser registrado (“Registered Ports”)
 - 49152 – 65535: Geral (“Dynamic and/or Private Ports”)

- ❑ Alguns dos serviços em well known ports

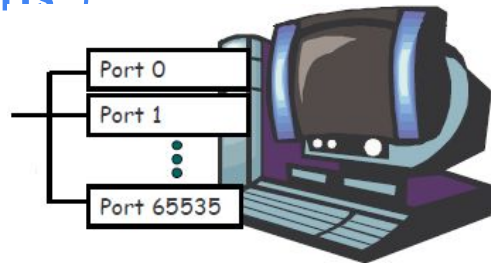
- DNS 53/tcp e 53/udp
- HTTP 80/tcp
- HTTPS 443/tcp
- SMTP 25/tcp
- POP3 110/tcp

- ❑ Alguns dos serviços em registered ports, em que cada servidor pode operar em diferentes portas configuráveis

- MySQL 3306/tcp
- Docker API 2375/tcp

- ❑ Dynamic and/or Private ports

- Sem registro: cada empresa usa para suas próprias aplicações; não há padronização ou necessidade de interoperabilidade global



Portas

❑ RFC 1700 (1994)

- Definiu inicialmente uma tabela estática com os serviços e portas

❑ Entre 1994 e 2002

- Não tinha como publicar uma nova RFC a cada atualização da RFC 1700, então decidiu-se que o ISI (Information Sciences Institute) e a IANA (Internet Assigned Numbers Authority) iriam disponibilizar atualizações online.

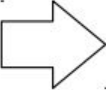
❑ RFC 3232 (2002):

- Oficializou o que ocorria desde 1994.
- Tornou a RFC 1700 obsoleta e passou a responsabilidade apenas para a IANA online:

<https://www.iana.org/assignments/service-names-port-numbers>

Associações (*bind*)

- Um processo precisa associar um socket a um endereço para “avisar” ao sistema operacional que deseja receber dados que chegam ao host com o endereço de destino especificado
- Um socket provê uma interface para enviar/receber dados para/da rede através de uma porta
 - Um endereço IP indica um pacote para uma máquina, enquanto que a **porta** permite a máquina decidir para que **processo/serviço** (se tiver algum) direcioná-lo.



Processos	Sockets	Endereço Associado
602	20	146.164.41.10 : 80
789	23	
102		
1026	25	146.164.41.10 : 21
	41	146.164.41.10 : 20

Processos	Sockets	Endereço Associado
602	20	146.164.41.10 : 80
789	23	146.164.41.10 : 56
102		
1026	25	146.164.41.10 : 21
	41	146.164.41.10 : 20

Entendendo a comunicação via socket: Cliente-Servidor

- ❏ Passos para escrever um cliente usando socket
 - Os passos que envolvem escrever uma aplicação cliente diferem superficialmente entre o TCP e o UDP.
 - Em ambos os casos:

- 1 Criar o *socket*
- 2 TCP: estabelecer a conexão com o servidor
- 3 Enviar algum dado para o servidor
- 4 Receber algum dado devolta.
- 5 *algumas vezes o envio e recepção podem alternar por um tempo*
- 6 TCP: fechar a conexão
- 7 Fechar o *socket*

Cliente-Servidor

□ Passos para escrever o Servidor

- Um servidor é um pouco mais complicado que o cliente, principalmente porque ele precisa estar apto a gerenciar vários clientes realizando conexões.
- Basicamente, existem dois aspectos em um servidor:
 1. Escutar por conexões de clientes
 2. Manipular cada conexão estabelecida
 - Enviar e receber mensagens do cliente

□ Pode-se separar o manipulador de uma conexão em uma função de suporte; e esta ser executada em *thread* separada.

API Sockets em C, mas similar em outras linguagens de programação

<code>socket()</code>	<i>Cria o socket. Retorna um "socket descriptor" (inteiro) ou -1 em caso de erro.</i>
<code>bind()</code>	<i>Conecta um socket (descriptor) em uma porta. Retorna 0 se sucesso e -1 se erro.</i> Obrigatório no servidor, TCP ou UDP. No cliente TCP, é o SO que escolhe a porta. No cliente UDP, é o SO que escolhe a porta, mas o usuário pode definir em casos de multicast, testes, firewalls e em protocolos que queiram, como P2P, RTP.
<code>listen()</code>	<i>Inicia a escuta na porta, definindo limite de conexões pendentes na fila de aceitação; não do número total de clientes que o servidor poderá atender ao longo do tempo.</i>
<code>accept()</code>	<i>Bloqueia até receber uma conexão. Retorna o socket descriptor de conexão com o cliente.</i>
<code>connect()</code>	<i>Faz uma conexão com um socket de escuta.</i> <i>Retorna 0 se sucesso.</i> Obrigatório no cliente TCP.
<code>read(), write(); recvfrom(), sendto(); recv(), send()</code>	<i>Funções usadas para transmitir e receber dados.</i>
<code>close(), shutdown()</code>	<i>Fecha o descritor do socket no processo. shutdown() desativa total ou parcialmente a comunicação em um socket conectado, permitindo, por exemplo, encerrar apenas o envio (SHUT_WR) e continuar recebendo dados.</i>

Sobre os exemplos em Python

- ❑ A criação de um socket é realizada via chamada à função `socket` com dois parâmetros
 - `AF_INET` (para indicar que o endereço IP é IPv4)
 - `SOCK_DGRAM` (UDP) ou `SOCK_STREAM` (TCP)
- ❑ Salientamos que, mesmo em Python, há muitos outros parâmetros para a criação de um socket. Consulte a documentação do Python para detalhes adicionais (<https://docs.python.org/3/library/socket.html#module-socket>)
- ❑ Em Python, uma string é representada por caracteres Unicode, que variam de 1 a 4 bytes, por exemplo, se a codificação for UTF-8. Então as chamadas a
 - `.encode()` → converte str (Unicode) em bytes (unidade de transmissão pela rede).
 - `.decode()` → converte bytes em str (Unicode).

Sobre os exemplos em Python

- ❑ `recvfrom` retorna uma tupla com
 - Mensagem do remetente (bytes recebidos)
 - (endereço IP, porta)
- ❑ `listen` vai indicar quantas conexões pendentes podem existir na fila do socket do servidor
 - Uma conexão pendente é aquela que já existe porque o handshaking já terminou, mas ainda não foi chamado `accept` para tirá-la da fila e torná-la ativa
 - Deve ter parâmetro maior ou igual a 1, e o máximo depende do SO, sendo tipicamente 128
 - Para ter um valor maior que 128 e máximo de 65535, deve-se utilizar, antes da chamada a `listen`, outra específica do SO, como `sysctl`

Programação com sockets usando UDP

UDP: não tem “conexão” entre cliente e servidor

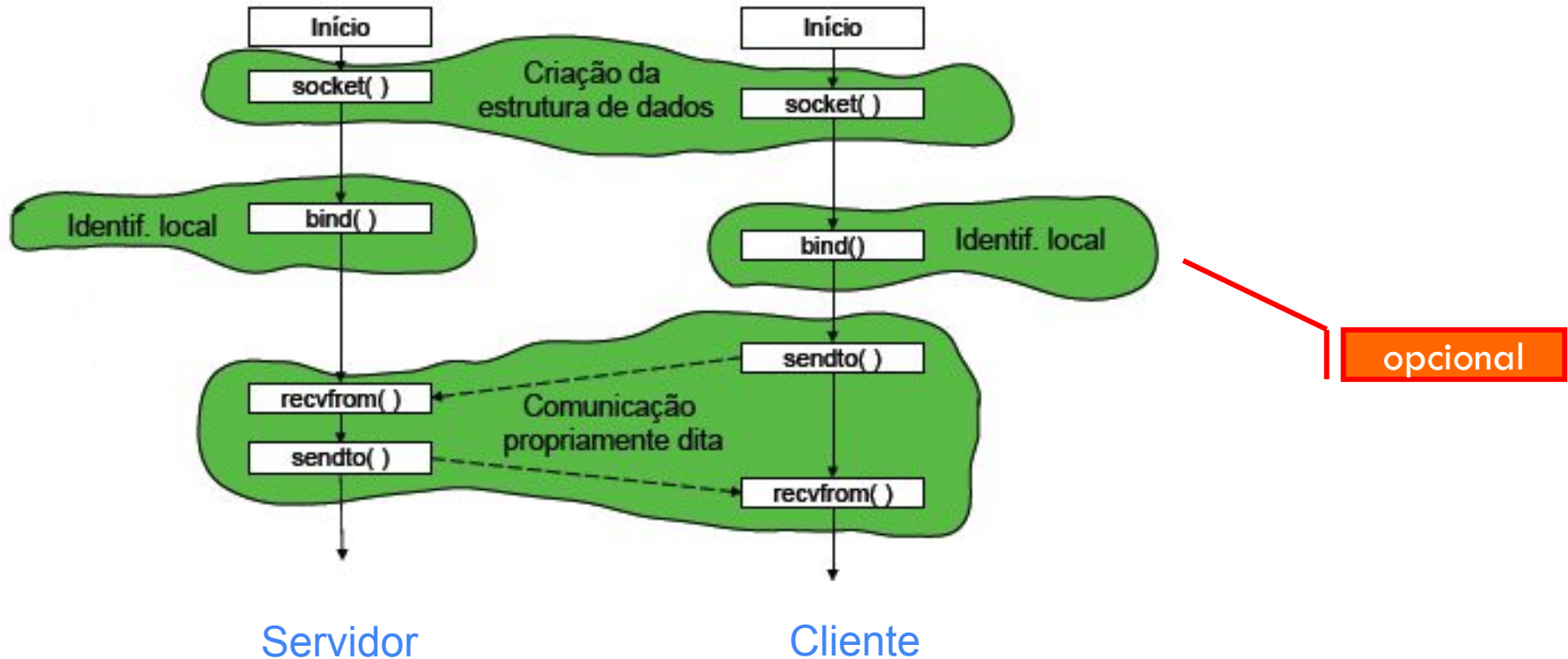
- ❑ não tem “handshaking”
- ❑ remetente coloca explicitamente endereço IP e porta do destino
- ❑ servidor deve extrair endereço IP, porta do remetente do datagrama recebido

UDP: dados transmitidos podem ser recebidos fora de ordem, ou perdidos

ponto de vista da aplicação

*UDP provê transferência
não confiável de grupos
de bytes (“datagramas”)
entre cliente e servidor*

Interface Sockets sem conexão - UDP



Exemplo de aplicação em Python

1. Cliente lê uma linha de caracteres (dados) de seu teclado e envia dados para o servidor
2. Servidor recebe os dados e converte caracteres em maiúsculas
3. Servidor envia dados modificados para o cliente
4. Cliente recebe dados modificados e exibe linha em sua tela

Exemplo app: cliente UDP

Python UDPClient

include Python's socket library	→	<code>from socket import *</code>
		<code>serverName = 'hostname'</code>
		<code>serverPort = 12000</code>
create UDP socket for server	→	<code>clientSocket = socket(AF_INET, SOCK_DGRAM)</code>
get user keyboard input	→	<code>message = input("Input lowercase sentence:")</code>
Attach server name, port to message; send into socket	→	<code>clientSocket.sendto(message.encode(), (serverName, serverPort))</code>
read reply characters from socket into string	→	<code>modifiedMessage, serverAddress = clientSocket.recvfrom(2048)</code>
print out received string and close socket	→	<code>print modifiedMessage.decode()</code> <code>clientSocket.close()</code>

Exemplo app: servidor UDP

Python UDPServer

```
from socket import *
serverPort = 12000

create UDP socket → serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port → serverSocket.bind(("", serverPort))
number 12000
print ("The server is ready to receive")

loop forever → while True:
    Read from UDP socket into → message, clientAddress =
    message, getting client's → serverSocket.recvfrom(2048)
    address (client IP and port) → modifiedMessage = message.decode().upper()
    send upper case string → serverSocket.sendto(modifiedMessage.encode(),
    back to this client → clientAddress)
```

Programação com sockets usando TCP

Cliente deve contatar servidor

- ❑ processo servidor deve antes estar em execução
- ❑ servidor deve antes ter criado socket (porta) que aguarda contato do cliente

Cliente:

- ❑ cria socket TCP local ao cliente
- ❑ especifica endereço IP, número de porta do processo servidor
- ❑ Quando cliente cria socket: TCP cliente deve criar conexão com TCP do servidor



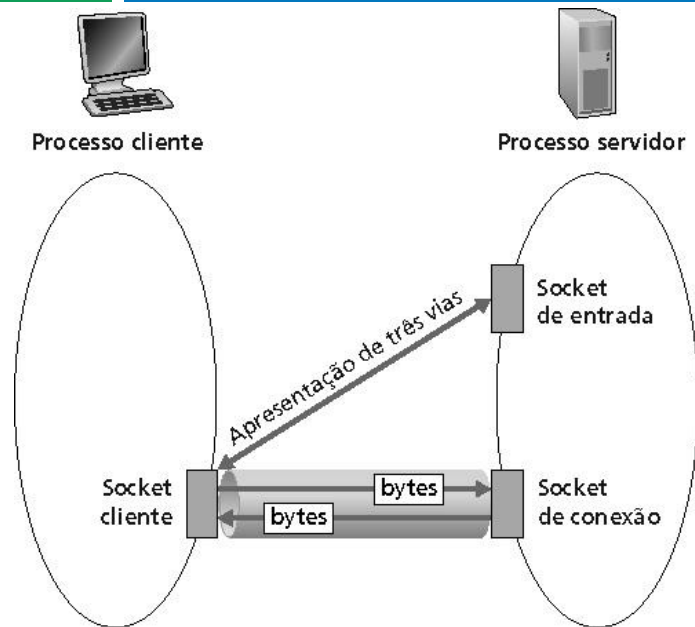
Quando contactado pelo cliente, o TCP do servidor cria socket novo para que o processo servidor possa se comunicar com o cliente

- permite que o servidor converse com múltiplos clientes
- Endereço IP e porta origem são usados para distinguir os clientes

ponto de vista da aplicação

TCP provê transferência confiável, ordenada de bytes (“tubo”) entre cliente e servidor

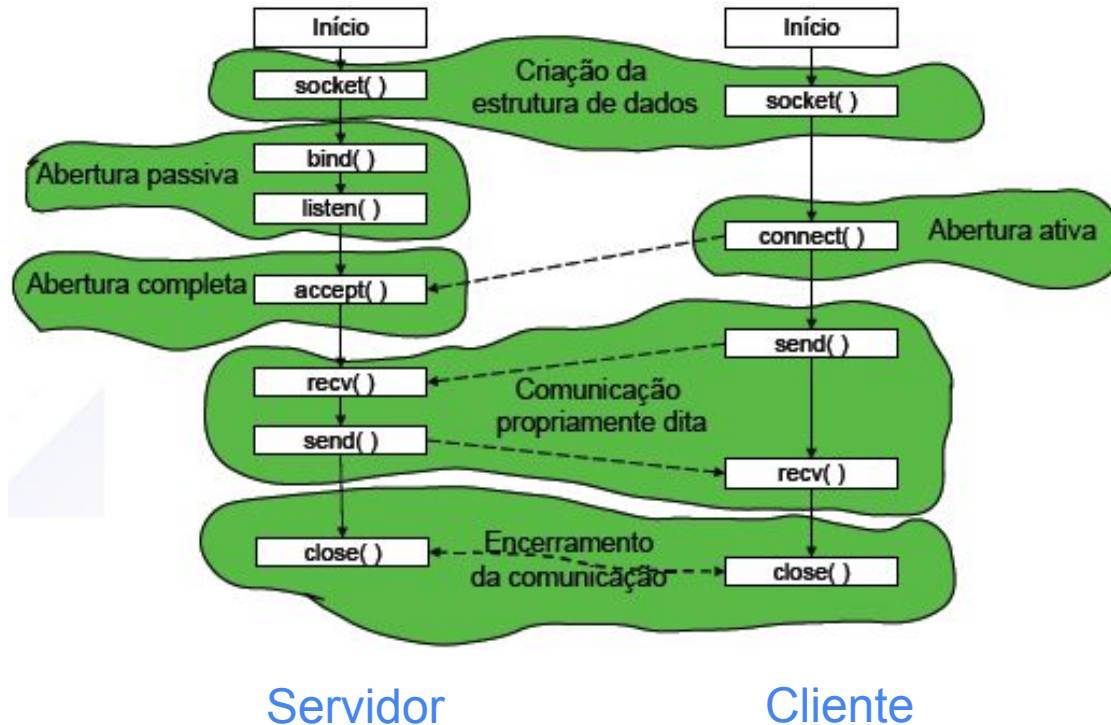
Comunicação entre sockets TCP



- ❑ O servidor tem uma porta conhecida para receber pedidos de conexão.
 - Depois que aceita o cliente, ele continua usando essa mesma porta do serviço,
 - mas em um novo socket que agora está amarrado à identidade daquele cliente específico, isto é, ao IP do cliente e à porta efêmera do cliente.

❑ Cada conexão aceita corresponde a uma **4-upla única**: (**IP cliente, porta cliente, IP servidor, porta servidor**). Isso permite que milhares de clientes diferentes conversem simultaneamente com o mesmo serviço, todos usando a mesma porta do lado do servidor.

Interface: Sockets com conexão - TCP



Estabelecimento de conexão

❏ Servidor

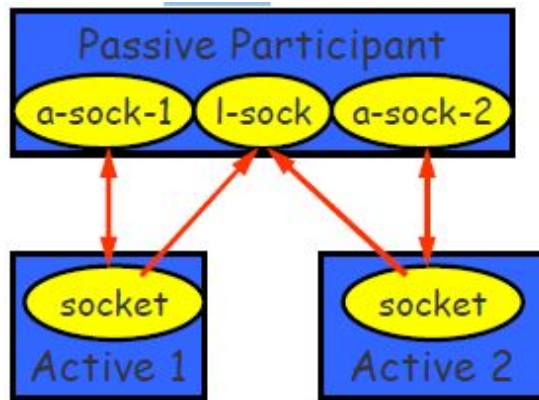
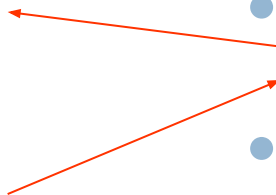
- passo 1: listen (por requisições)
- passo 3: accept (uma requisição)
- passo 4: transf. de dados

❏ A conexão aceita fica em um novo socket

- O socket antigo continua esperando novos participantes

❏ Cliente

- passo 2: requisição & estabelecimento de conexão
- passo 4: transf. de dados



Exemplo app: cliente TCP

Python TCPClient

```
from socket import *
```

```
serverName = 'serverIP'
```

create TCP socket for
server, remote port 12000

```
serverPort = 12000
```

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

```
clientSocket.connect((serverName, serverPort))
```

```
sentence = input("Input lowercase sentence: ")
```

No need to attach server
name, port

```
clientSocket.send(sentence.encode())
```

```
modifiedSentence = clientSocket.recv(1024)
```

```
print ("From Server: ", modifiedSentence.decode())
```

```
clientSocket.close()
```

Exemplo de app: servidor TCP

Python TCPServer

create TCP welcoming socket	→	<code>from socket import *</code>
server begins listening for incoming TCP requests	→	<code>serverPort = 12000</code>
	→	<code>serverSocket = socket(AF_INET,SOCK_STREAM)</code>
	→	<code>serverSocket.bind("",serverPort)</code>
loop forever	→	<code>serverSocket.listen(1)</code>
server waits on accept()	→	<code>print ("The server is ready to receive ...")</code>
for incoming requests, new socket created on return	→	<code>while True:</code>
	→	<code>connectionSocket, addr = serverSocket.accept()</code>
read bytes from socket (but not address as in UDP)	→	<code>sentence = connectionSocket.recv(1024).decode()</code>
	→	<code>capitalizedSentence = sentence.upper()</code>
close connection to this client (but <i>not</i> welcoming socket)	→	<code>connectionSocket.send(capitalizedSentence.encode())</code>
	→	<code>connectionSocket.close()</code>

Atenção para uma Armadilha!

❑ As funções

`read`, `recv`, `recvfrom`, `write`, `send` e `sendto` podem retornar menos bytes do que a quantidade requisitada, e nenhuma mensagem de erro é retornada neste caso!

❑ Solução: ao usar as funções `recv`, `recvfrom`, `send` e `sendto` pode-se passar como parâmetro a flag `MSG_WAITALL`.

Close

- ❑ Após a utilização, um socket deve ser fechado:
- ❑ `status = close(s);`
 - `status`: 0 se Ok, -1 se erro
 - `s`: descritor (socket sendo fechado)
- ❑ fechando um socket
 - fecha-se uma conexão (SOCK_STREAM)
 - libera a porta utilizada pelo socket

Ordem de bytes para endereços e portas

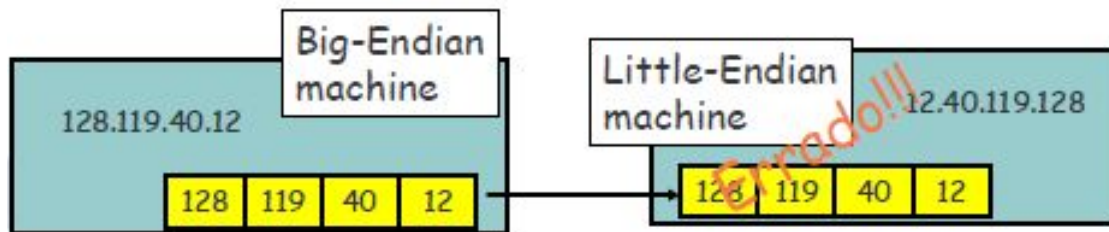
Endereços e portas são armazenados como inteiros

- `u_short sin_port;` (16 bit)
- `in_addr sin_addr;` (32 bit)

```
struct in_addr {  
    u_long s_addr;  
};
```

Problema:

- máquinas/SOs diferentes usam diferentes ordens
 - little-endian: lower bytes first
 - big-endian: higher bytes first
- essas máquinas podem se comunicar em uma rede



Solução: Ordem de Rede

❑ Definições:

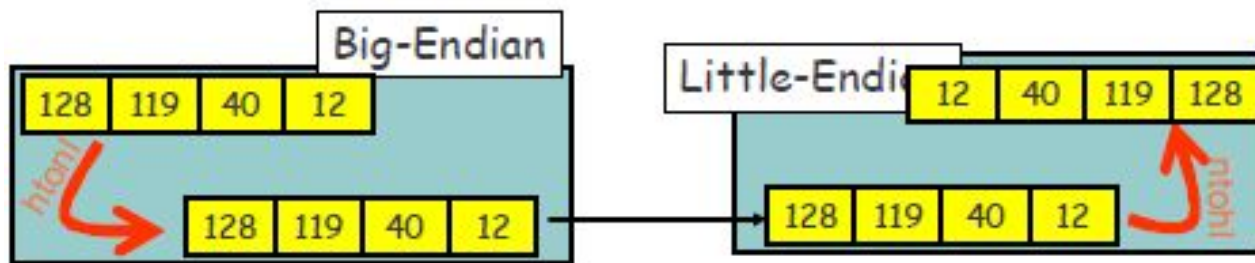
- **Host Byte-Ordering:** a ordem utilizada por um host (big ou little)
- **Network Byte-Ordering:** a ordem utilizada pela rede – sempre **big-endian**

❑ Qualquer palavra enviada pela rede deve ser convertida para a ordem de rede antes da transmissão (e de volta para a ordem do host quando recebida)

❑ *P:* Uma vez que máquinas *big-endian* não precisam de conversão e máquinas *little-endian* precisam, como podemos evitar duas versões diferentes de um mesmo programa?

Funções de ordenação de bytes

- u_long **htonl**(u_long x);
- u_short **htons**(u_short x);
- Em máquinas big-endian, essas rotinas não fazem nada
- Em máquinas little-endian, elas invertem a ordem dos bytes



Prefixos das funções:
hton: host do network
ntoh: network to host

As funções `encode()` e `decode()` em Python tratam apenas da conversão entre texto e bytes, não da ordem de bytes de valores numéricos. A conversão para network byte order ocorre em funções específicas, como `socket.htons()` ou `struct.pack()`.

Lidando com chamadas bloqueantes

- ❑ Muitas funções que vimos ficam bloqueadas até que algum evento aconteça
 - `accept`: até que uma conexão seja iniciada
 - `connect`: até que a conexão seja estabelecida
 - `recv`, `recvfrom`: até que um pacote seja recebido
 - `send`, `sendto`: até que os dados sejam transferidos para um buffer do socket
- ❑ Para programas simples, bloqueio pode ser conveniente
- ❑ E no caso de programas mais complexos?
 - Conexões múltiplas
 - Envio e recebimento simultâneo
 - Processamento interno simultaneamente com comunicação de rede

Lidando com chamadas bloqueantes

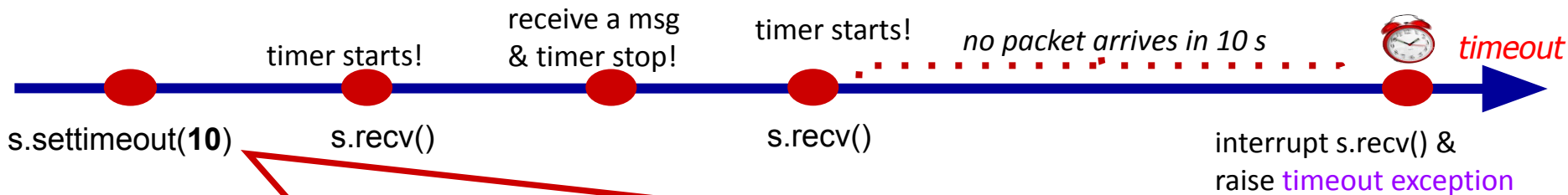
Opções:

- Criar código multi-threaded ou multi-processo (fork)
- Desabilitar o bloqueio (por ex., utilizando a função fcntl)
- Utilizar multiplexação de E/S, com a função select()
- Limitar o tempo de bloqueio (*timeout*), usando:
 - i. setsockopt()
 - `level = SOL_SOCKET, optname = SO_RCVTIMEO ou SO_SNDTIMEO`
 - ii. settimeout(), ou
 - iii. através de sinais do SO

OBS: Outras opções de setsockopt() - `SO_BROADCAST, SO_REUSEADDR`
(reuso assim que o socket é fechado em UNIX/Linux)

Exemplo em Python

`socket.settimeout()`



Define um tempo limite em todas as futuras operações desse soquete específico!

 **Bloco `try-except`:** Executa um bloco de código e lida com “exceções” que podem ocorrer ao executar esse bloco de código

`try:`

< Faça algo >

`except <exception>:`

< Trata a exceção >

{ Se ao executar esse bloco, uma exceção for gerada, a execução salta diretamente para o bloco de código **`except`**

{ Este código só é executado se ocorrer uma **`<exception>`** no bloco de código **`try`**. Exemplo:

`try:`

...

`except timeout:`

Referências

- Stevens, W. R.; Fenner, B.; Rudoff, A. M. Programação de redes UNIX - API para Soquetes de Redes- Vol. 1. 3. ed. São Paulo: Bookman, 2005.
- Laureano, M. Programando em C para Linux, Unix e Windows, Rio de Janeiro: Brasport, 2005.
- Stevens, W. R. UNIX Network Programming Volume 2: Interprocess Communications. 2. ed. Upper Saddle River: Prentice Hall PTR, 1998.

